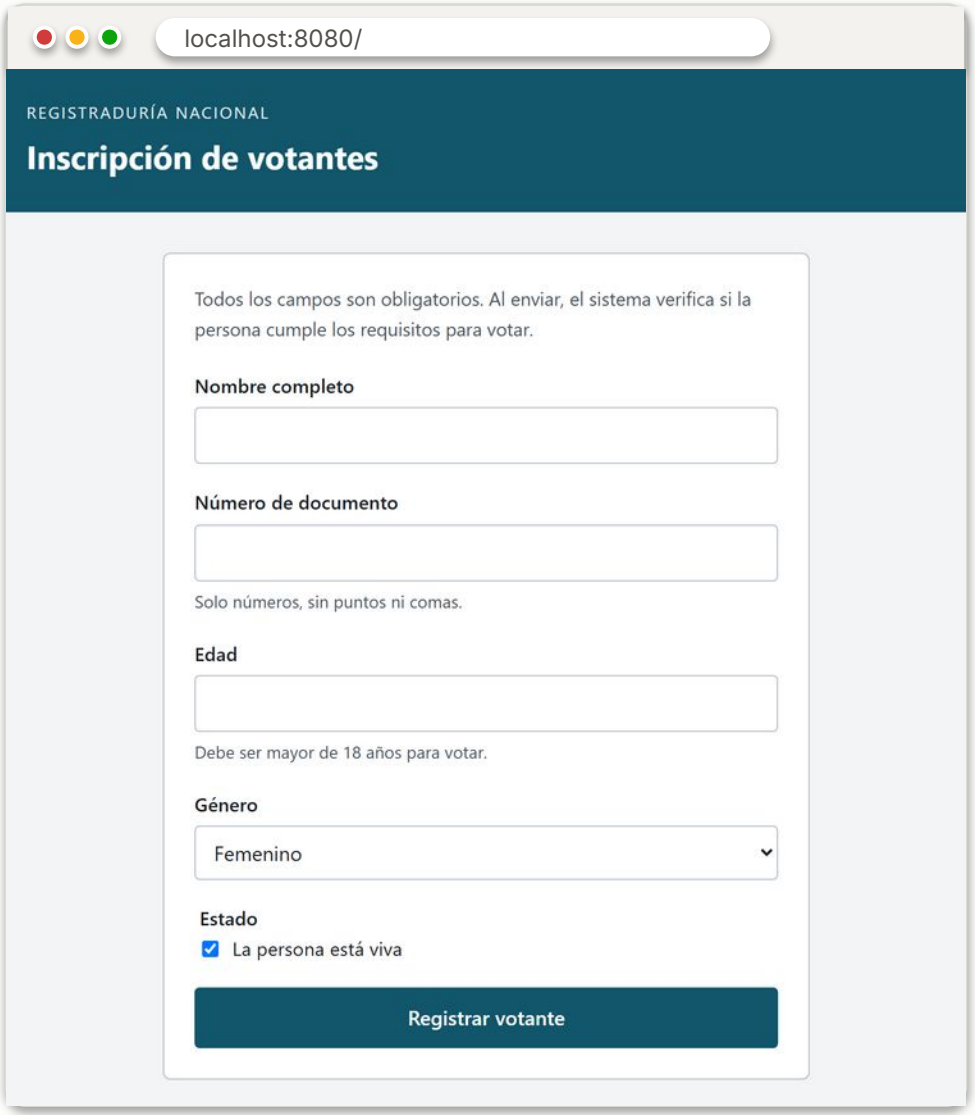
TESTING Y VALIDACIÓN DE SOFTWARE

Pruebas de UI y UX

¿La interfaz funciona? Eso lo responde una máquina. ¿La gente entiende cómo usarla? Eso solo se responde mirando a personas.

E2E | Page Object Model | Accesibilidad | Regresión visual | Usabilidad | Probar con IA

Sistema bajo prueba: la Registraduría (Spring Boot), la misma aplicación de los talleres de pruebas unitarias, de integración y de carga.



MAPA DE LA SESIÓN

Siete módulos, cinco preguntas distintas

1

Pruebas E2E

¿La interfaz funciona?

9 pruebas

2

Page Object Model

¿Las pruebas sobreviven a un rediseño?

5 pruebas

3

Accesibilidad

¿Cumple lo que una máquina puede medir de WCAG?

5 pruebas

3B

Ver fallar a axe

¿Qué no ve axe? 17 defectos sembrados

6 pruebas

4

Regresión visual

¿Cambió algo sin querer?

3 pruebas

5

Usabilidad con usuarios

¿La gente entiende cómo usarla?

sin automatizar

6

Probar con IA

¿Sirve una IA para probar? Se mide, no se opina

exploratorio · 0 pts

→

Entrega

Repositorio, Wiki, CI y reflexión final

rúbrica de 65 pts

Automatizar responde cuatro preguntas. La quinta, no

UI · lo mide una máquina ejecutando pruebas

UX · observando personas

M1

E2E

¿La interfaz funciona?

M2

Page Objects

¿Las pruebas sobreviven a un rediseño?

M3 · 3B

Accesibilidad

¿Cumple lo que una máquina puede medir de WCAG?

M4

Regresión visual

¿Cambió algo sin querer?

M5

Usabilidad

¿La gente entiende cómo usarla?

Un formulario puede pasar las 28 pruebas automatizadas y aun así hacer que la gente abandone en el segundo campo. La usabilidad se mide observando a personas, no ejecutando código.

E2E (end to end, de extremo a extremo): probar el sistema completo como lo usaría una persona, del navegador al servidor.

SISTEMA BAJO PRUEBA

La regla de negocio vive en dos lugares

Persona en el navegador

Llena el formulario y pulsa "Registrar votante".

Capa 1 · app.js en el navegador

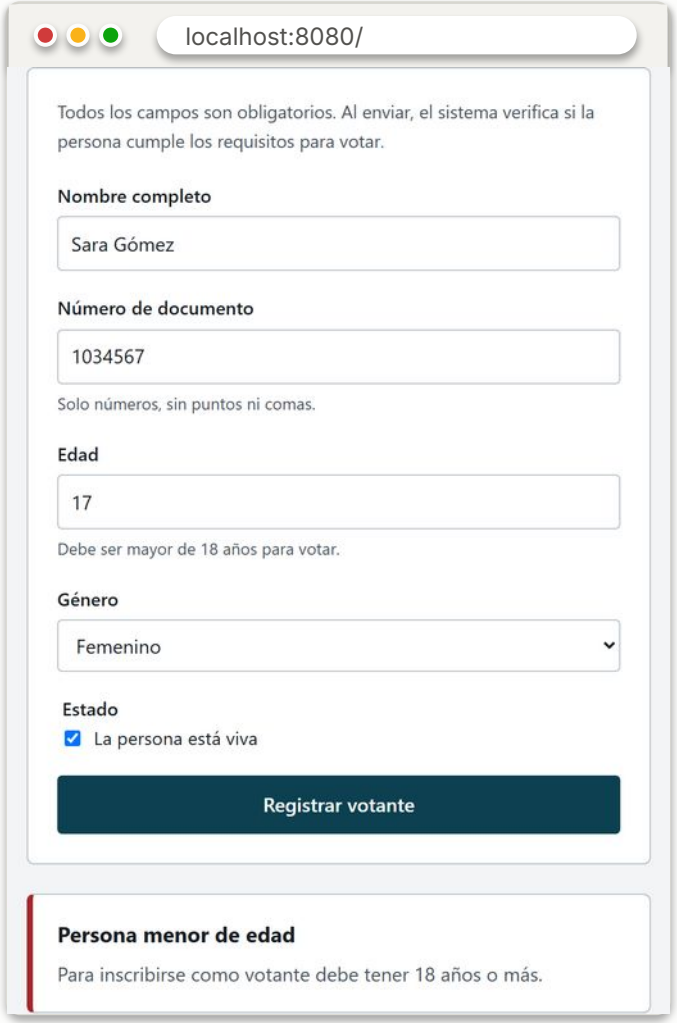
Valida nombre, documento y edad entre 0 y 120. Si falla, muestra el error y no llama al servidor.

Capa 2 · POST /register (Spring Boot)

Valida documento, vida, edad y duplicados, en ese orden, y responde VALID o el motivo: INVALID, DEAD, INVALID_AGE, UNDERAGE o DUPLICATED.

Resultado en pantalla

Con role="status" (ARIA), el lector de pantalla, el programa que lee en voz alta, lo anuncia.



Playwright: un navegador controlado por código

1 Qué es

Una librería de Microsoft que abre un navegador real (aquí, Chromium) y lo maneja desde JavaScript.

2 Qué es page

La pestaña del navegador. Cada prueba recibe una nueva y limpia, sin lo que dejó la anterior.

3 Cómo corre

npm test ejecuta todas las pruebas. Antes, webServer levanta la Registraduría y espera a que responda.

Para verlo trabajar: `npm run test:headed`. Para ir paso a paso: `npm run test:ui`.

Una prueba real: la 01 del módulo 1

```
test.beforeEach(async ({ page }) => {
  await page.goto('/');        // http://localhost:8080/
});

test('01 - La página carga con el título correcto',
  async ({ page }) => {
    await expect(page).toHaveTitle(/Registraduría/);
    await expect(page.getByRole('heading',
      { name: 'Inscripción de votantes' })).toBeVisible();
  });
```

`test(...)`	una prueba: un escenario con su nombre
`page.getByRole(...)`	un localizador: cómo se encuentra un elemento
`expect(...)`	una aserción: lo que debe cumplirse; se reintenta sola

Buscar los elementos como los busca una persona

Bien

```
page.getByRole('button', { name: 'Registrar votante' })
```

Sobrevive a un rediseño y, de paso, comprueba que el botón es accesible.

Mal

```
page.locator('.btn.btn-primary.mt-3')
```

Se rompe con el primer cambio de estilos.

Peor

```
page.locator('/html/body/div[2]/form/button')
```

XPath absoluto: se rompe si alguien agrega un div. Lo penaliza la rúbrica.

Orden de preferencia de la guía: rol → etiqueta → texto → test id → CSS o XPath, como último recurso.

Una prueba inestable casi nunca es mala suerte

Cero esperas fijas

```
// Una apuesta: lento si responde rápido,
// insuficiente si responde lento
await page.waitForTimeout(3000);

// Reintenta hasta que se cumple o expira
await expect(page.getByRole('heading',
  { name: 'Inscripción exitosa' })).toBeVisible();
```

Un sleep es la causa número uno de pruebas inestables. La entrega exige cero, y la rúbrica las penaliza junto con el XPath absoluto.

MEDIDO EN ESTE REPOSITORIO

1 de cada 30

ejecuciones fallaba con "Documento ya inscrito". Medido con --repeat-each=30 (cada prueba 30 veces) y 8 workers (8 procesos en paralelo).

```
// Antes: dos copias, mismo milisegundo
Date.now() % 1000000
// Ahora: 900 millones de valores
Math.floor(Math.random() * 900_000_000)
```

Para cazar una prueba inestable, repítala en paralelo: --repeat-each es la forma estándar.

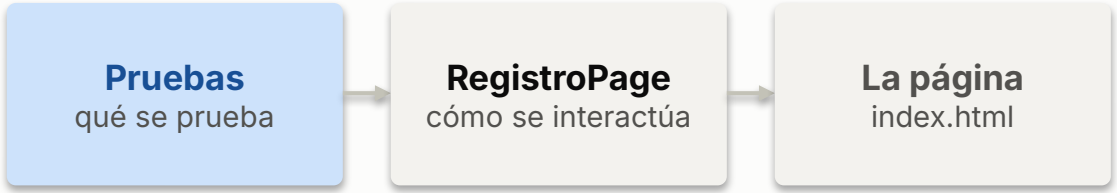
Las pruebas se leen como reglas de negocio

Sin Page Object: una secuencia de clics

```
await page.getByLabel('Nombre completo').fill('Ana');
await page.getByLabel('Número de documento')
  .fill(String(documento));
await page.getByLabel('Edad').fill('17');
await page.getByRole('button',
  { name: 'Registrar votante' }).click();
await expect(page.getByRole('heading',
  { name: 'Persona menor de edad' })).toBeVisible();
```

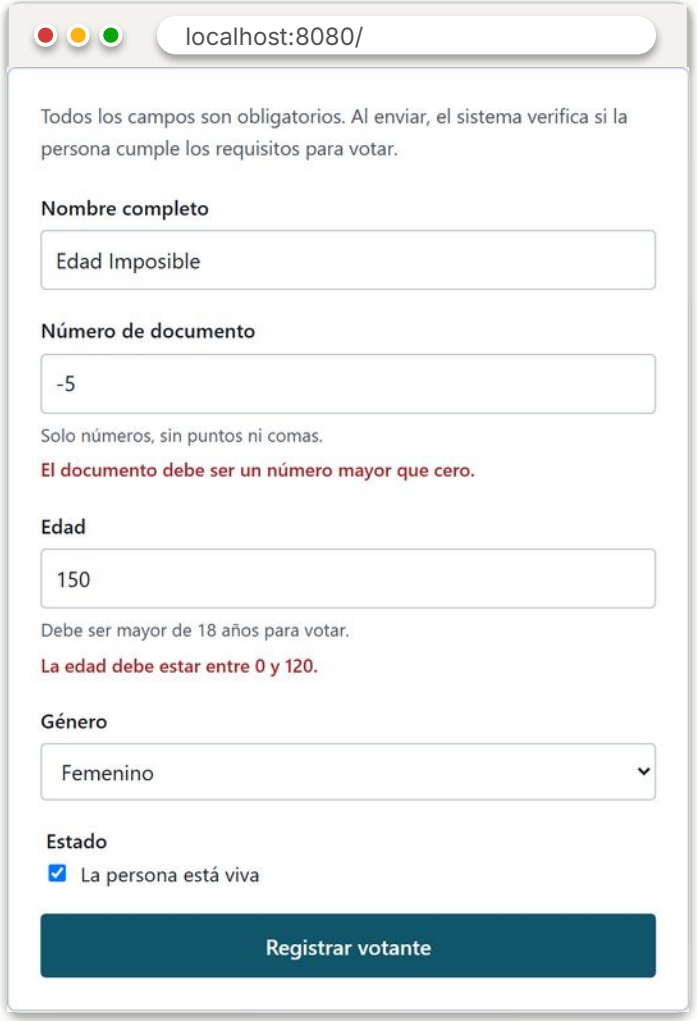
Con Page Object: la regla que se prueba

```
await registro.inscribir({
  documento: RegistroPage.documentoUnico(), edad: 17 });
await registro.esperarResultado('Persona menor de edad');
```



Cuando la interfaz cambia, se corrige en un solo lugar y las pruebas siguen funcionando. Los campos que la prueba no declara (nombre, género, "está viva") toman valores por defecto razonables: cada prueba dice solo lo que le importa.

La prueba 09: una regla que la interfaz no deja alcanzar



Capa 1 · el navegador detiene el caso

```
await page.getByLabel('Edad').fill('150');
...click();
await expect(page.getByText('La edad debe estar entre 0 y
120.')).toBeVisible();
expect(huboLlamada).toBe(false);   // nunca llegó al servidor
```

Capa 2 · la API sí es alcanzable sin el formulario

```
const r = await request.post('/register',
  { data: { ..., age: 150 } });
expect((await r.text()).trim()).toBe('INVALID_AGE');
```

Una suite E2E mide lo que se alcanza a través de la interfaz. Eso no es lo mismo que lo que el sistema hace.

axe: un revisor automático de accesibilidad

1 Qué es

Un motor de código abierto (axe-core, de Deque) que revisa la página contra reglas de WCAG.

2 Cómo se usa aquí

Playwright abre la página y @axe-core/playwright se la pasa a axe para que la analice.

3 Qué devuelve

Una lista de violaciones: la regla, qué tan grave es, cómo corregirla y en qué elemento está.

La prueba del módulo 3

```
const resultados = await new AxeBuilder({ page })
  .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'])
  .analyze();

expect(resultados.violations).toEqual([]);
```

Lo que imprime si falla (salida real sobre defectuosa.html)

```
[critical] image-alt: Images must have alternative text
  https://dequeuniversity.com/rules/axe/4.13/image-alt
    img:nth-child(2)
```

La prueba pasa solo si la lista viene vacía: 0 violaciones en index.html, 6 reglas en defectuosa.html. Axe responde en inglés.

WCAG (Web Content Accessibility Guidelines, del W3C) es la norma internacional de accesibilidad web. Tiene versiones (2.0, 2.1) y niveles: A es lo mínimo, AA es el que suelen exigir las leyes y AAA el más estricto. Las etiquetas del código se leen así: **`wcag21aa`** = WCAG 2.1, nivel AA.

Sin violaciones de axe no es lo mismo que accesible

≈ 40 %

de los problemas WCAG los detecta axe de forma fiable. El resto exige juicio humano.

European Accessibility Act: exigible desde junio de 2025. En Estados Unidos, la ADA genera litigio constante sobre sitios web.

Lo que mide una máquina

- Contraste insuficiente
- <img> sin alt
- Botones y enlaces sin nombre accesible
- Página sin idioma declarado
- Un <select> sin nombre

Lo que exige juicio

- Si el texto alternativo es útil
- Si el foco de teclado se ve
- Si un enlace dice a dónde lleva
- Si un error explica qué corregir
- Si el resultado se anuncia

En verde significa otra cosa: "no tiene los errores que una máquina puede detectar sola". Por eso el módulo audita también la página con errores de validación visibles, el punto ciego clásico.

17 defectos sembrados, en tres montones

`defectuosa.html` es la misma pantalla del formulario con 17 defectos de accesibilidad puestos a propósito. Como se sabe cuáles son, se puede medir qué encuentra axe.

Grupo A 7 defectos	Grupo B 3 defectos	Grupo C 7 defectos
6 reglas	**4 reglas**	**revisión manual**
axe los reporta con .withTags([...wcag...])	axe los reporta solo si se quita ese filtro	axe no los reporta nunca

button-name: botón sin nombre
color-contrast: poco contraste
html-has-lang: sin idioma
image-alt: imagen sin alt
link-name: enlace sin texto
select-name: lista sin etiqueta
	`heading-order`
	`landmark-one-main`
	`region`
	`tabindex`
		placeholder como única etiqueta, alt="imagen", foco invisible, casilla "Estado", resultado sin role="status", "Haga clic aquí", un error que dice solo "Error."

Axe encuentra 10 de 17. Ninguno de los 7 del grupo C: esos los localiza usted, a mano, y explica por qué ninguna herramienta puede detectarlos.

Una línea de configuración esconde cuatro defectos reales

Con filtro: 6 reglas

```
new AxeBuilder({ page })
  .withTags(['wcag2a', 'wcag2aa', 'wcag21a',
'wcag21aa'])
  .analyze();
```

- button-name
- color-contrast
- html-has-lang
- image-alt
- link-name
- select-name

Qué señalan las 4 reglas escondidas

`heading-order`	los títulos saltan de h1 a h4
`landmark-one-main`	la página no tiene <main>
`region`	hay contenido fuera de toda región
`tabindex`	el foco no sigue el orden visual

Sin filtro: 10 reglas

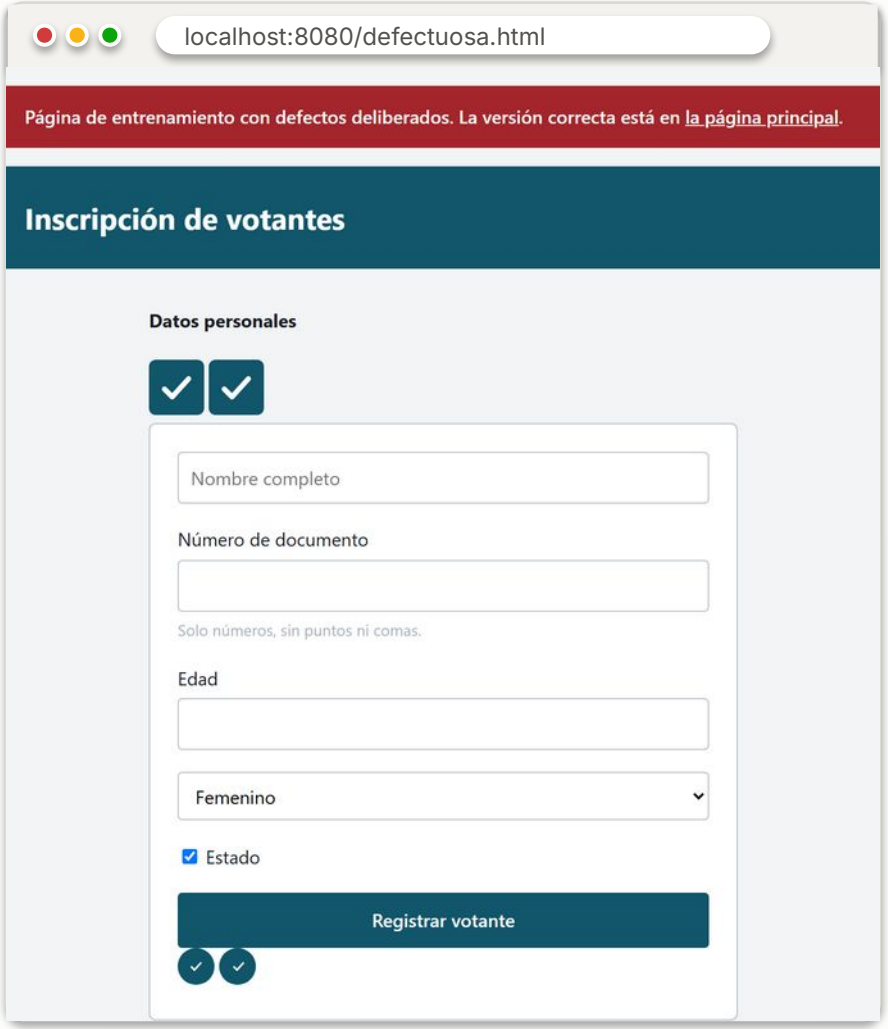
```
new AxeBuilder({ page })

  .analyze();
```

- button-name
- color-contrast
- html-has-lang
- image-alt
- link-name
- select-name
- heading-order
- landmark-one-main
- region
- tabindex

En naranja, best-practice: reglas que axe recomienda aunque WCAG no las pida de forma literal. El filtro WCAG las silencia.

Axe aprueba un campo sin etiqueta



El placeholder (el texto gris dentro del campo) **cuenta como nombre accesible,** el nombre que el lector de pantalla anuncia para el campo. Por eso axe lo da por etiquetado. Pero desaparece en cuanto la persona empieza a escribir.

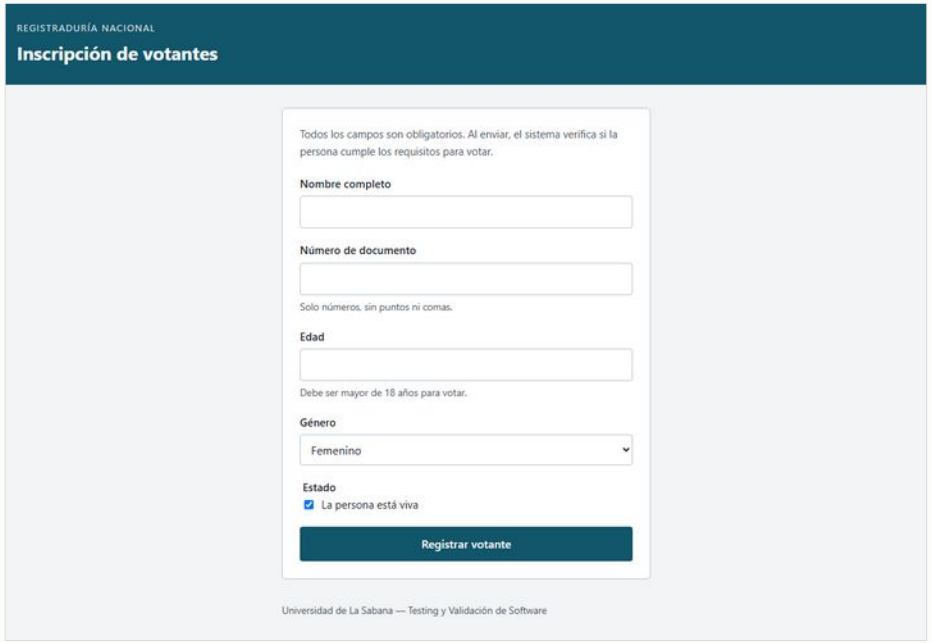
Los 7 defectos que ninguna herramienta ve

- **placeholder usado como única etiqueta**
- alt="imagen": existe, pero no describe nada
- outline: none, el foco de teclado se vuelve invisible
- una casilla rotulada "Estado"
- el resultado sin role="status": nunca se anuncia
- un enlace que dice "Haga clic aquí"
- un error que dice "Error." y nada más (WCAG 3.3.3)

La prueba 05 del módulo 3, escrita a mano, sí lo encuentra.

Lo que ninguna aserción sobre el DOM ve

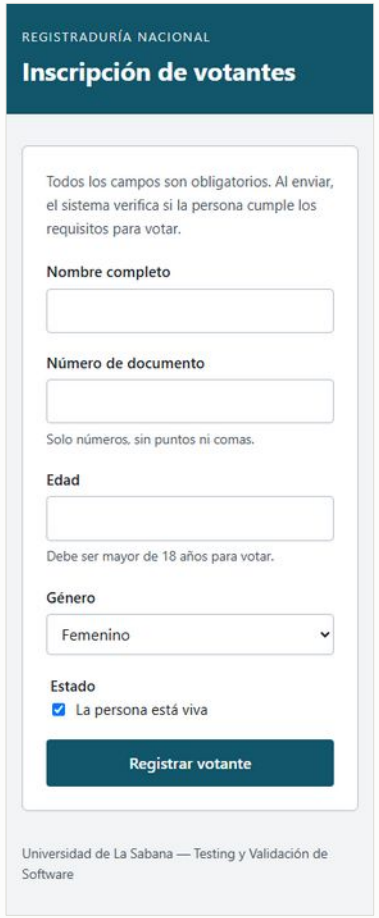
formulario-vacio · 1280 px



resultado-rechazo · solo el mensaje



móvil · 375 px




```
npm run test:visual     # comparar
npm run visual:update   # actualizar
```

El riesgo: actualizar las referencias (las capturas aprobadas) sin mirar el diff, la imagen que marca los píxeles que cambiaron. Desde ese momento la prueba aprueba cualquier cosa.

Dependen del sistema operativo: una referencia tomada en Windows no coincide con una tomada en Linux, donde corre la integración continua (CI). Por eso el CI del taller no ejecuta este módulo.

Tareas como objetivos, no como instrucciones

5 participantes detectan cerca del 85 % de los problemas

Nielsen. Personas que no hayan trabajado en el proyecto.

INSTRUCCIÓN · NO SIRVE

"Escribe 30 en el campo Edad y pulsa Registrar"

OBJETIVO · SIRVE

"Inscribe a tu tía Ana, de 30 años, para que pueda votar"

La primera ya contiene la respuesta: solo comprueba que los dedos funcionan. La segunda revela si la persona encuentra el camino.

Tasa de éxito

% que la completa sin ayuda

Tiempo en tarea

segundos hasta completarla

Errores

acciones equivocadas antes de acertar

Abandono

% que se rinde

"No ayudes." Si alguien no encuentra el botón en 30 segundos, ese silencio incómodo es el hallazgo. Pida pensar en voz alta y anote lo que hacen, no solo lo que dicen.

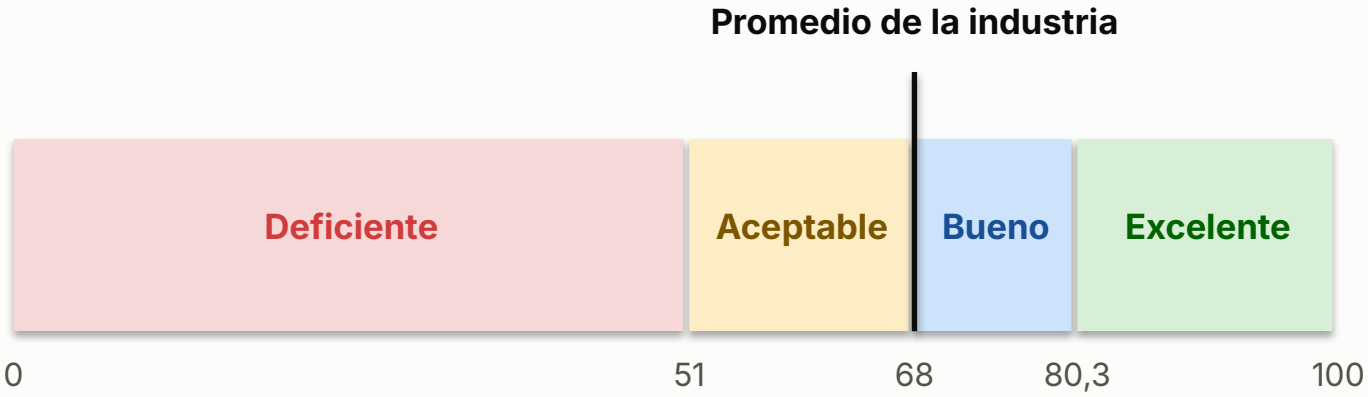
SUS: 68 no es "68 %", es exactamente el promedio

SUS (System Usability Scale): 10 afirmaciones estándar que cada participante califica de 1 a 5 al terminar. Mide la usabilidad que la persona percibió, no lo que hizo. Se alternan: *"Pensé que el sistema era fácil de usar"* (impar, positiva) y *"Encontré el sistema innecesariamente complejo"* (par, negativa).

Cómo se calcula

1. Impares (1, 3, 5, 7, 9): **`x − 1`**
2. Pares (2, 4, 6, 8, 10): **`5 − x`**
3. Sume los 10 valores y **`multiplique por 2,5`**

Resultado de 0 a 100. No es un porcentaje: es una posición frente a otros sistemas medidos.



Ejemplo: un promedio de 72,5 queda en "bueno", apenas por encima de la media. Un 50 no es "la mitad de bien": es peor que la mayoría de los sistemas medidos.

Bandas de Sauro y Lewis. Más de 80,3 corresponde al percentil 90.

MÓDULO 6 · PROBAR CON IA · EXPLORATORIO

A la IA no se le cree: se la evalúa

1 **Prompt**
plantilla del taller

→

2 **Ejecutar**
ChatGPT o Claude Code

→

3 **Salida**
JSON o código

→

4 **Evaluar**
script contra una verdad conocida

→

5 **Reporte**
HTML que explica qué pasó

E1 Auditoría

¿Encuentra la IA lo que axe no ve? Los 7 del grupo C.

E4 Estabilidad

¿Encuentra lo mismo en tres ejecuciones con el mismo prompt?

E2 Pruebas generadas

¿Detectan algo cuando se rompe la aplicación a propósito?

E3 Plan exploratorio

¿Qué reglas descubre un agente usando la app? ¿Caja negra o blanca?

Sesión 1: E1 y E4, auditar defectuosa.html. **Sesión 2:** E2 y E3, generar pruebas y diseñar un plan. **No suma puntos en la rúbrica.**

Si la IA puede ver las respuestas, no las encuentra: las copia

DÓNDE ESTÁN LAS RESPUESTAS	ARRUINA	CÓMO SE EVITA
Los comentarios de defectuosa.html y el módulo 3B	E1 · E4	`npm run ia:preparar` copia sin comentarios, fuera del repositorio
Las pruebas de los módulos 1 y 2	E2	`npm run ia:preparar-generacion` una carpeta sin pruebas
El README y las pruebas del taller	E3	**caja negra: carpeta vacía** caja blanca: solo registraduria/

En aprendizaje automático esto tiene nombre: evaluar un modelo con datos que ya vio. El resultado siempre sale espectacular y nunca significa nada.

Que una prueba pase no dice si protege

Es la idea de las pruebas de mutación: un proxy se pone delante de la Registraduría y altera sus respuestas para meter un defecto a la vez. Si alguna prueba falla, lo detectó.

	Suite del taller	Ejemplo con defectos típicos
S1 · Se inscriben menores de edad	Detectado	Detectado
S2 · Se aceptan documentos repetidos	Detectado	Detectado
S3 · Se inscriben personas fallecidas	Detectado	No ejercitado
S4 · La API acepta edades imposibles	Detectado	No ejercitado
S5 · Desaparece la validación del navegador	Detectado	Detectado
S6 · El resultado deja de anunciarse	Sobrevivió	Sobrevivió
	5 de 6	**3 de 6**

Sobrevivió es una aserción débil. **No ejercitado** es un caso que no existe. **Las dos suites pasan en verde con la aplicación intacta.**

HERRAMIENTAS

Playwright o Selenium: la misma aplicación, dos pistas

	Playwright	Selenium
Espera automática	Sí	Manual: se pide con WebDriverWait
Levanta la aplicación	Sí, con webServer en la configuración	No: hay que arrancarla antes
Accesibilidad	@axe-core/playwright	Requiere integración
Regresión visual	Nativa	Librería externa
Lenguajes	JS/TS, Python, Java, .NET	Prácticamente todos
Industria	En crecimiento	Estándar histórico
En este taller	Pista principal · 28 pruebas	Pista alternativa · 7 pruebas

ENTREGA

Cómo se califica la entrega



65 puntos

13 criterios × 5. E2E, accesibilidad y usabilidad valen por dos.

59 – 65	Excelente dominio técnico y metodológico
46 – 58	Buen trabajo, análisis parcial
39 – 45	Cumple lo básico, sin profundidad
< 39	No cumple los mínimos

Antes de entregar: clone su propio repositorio en otra carpeta y ejecute npm test. Si no queda en verde sin pasos manuales, falta algo.

LA PREGUNTA QUE CIERRA EL TALLER

¿Qué problema encontraron los usuarios que ninguna de las 28 pruebas automatizadas podía detectar?

- ¿Qué encontró axe que usted no habría notado mirando la pantalla?
- ¿Qué le costó más: escribir las pruebas o mantenerlas estables?
- Si hizo el módulo 6: ¿para qué tarea usaría una IA al probar, para cuál no, y qué verificación manual no se saltaría nunca?